

Express Authentication & API Documentation

1. CryptoJS

1.1 What is CryptoJS?

CryptoJS is a JavaScript library used for:

- Hashing passwords
- Encrypting and decrypting data
- Improving application security

Passwords must **never be stored in plain text**. Instead, they are hashed before storing in the database.

1.2 Why Hash Passwords?

- Hashing is a **one-way process**
 - Even if the database is compromised, original passwords cannot be retrieved
 - Industry standard practice
-

1.3 Installation

```
npm install crypto-js
```

1.4 Importing CryptoJS

```
const cryptojs = require('crypto-js')
```

1.5 Hashing Password Using SHA256

```
const hashedPassword = cryptojs.SHA256(password).toString()
```

2. Signup API

2.1 What is Signup?

Signup allows a new user to:

- Register into the system
 - Store personal details
 - Securely store password in hashed form
-

2.2 Signup Flow

Client → Request Body → Hash Password → Store in Database

2.3 Signup API Code

```
router.post('/signup', (req, res) => {  
  const { name, email, password, mobile } = req.body  
  
  const hashedPassword = cryptojs.SHA256(password).toString()  
  
  const sql = `INSERT INTO users(name,email,password,mobile) VALUES  
  (?, ?, ?, ?)`  
  
  pool.query(sql, [name, email, hashedPassword, mobile], (error, data)  
=> {  
    res.send(result.createResult(error, data))  
  })  
})
```

2.4 Key Points

- Password is hashed before storing
 - Plain password is never saved
 - Uses `req.body` to receive data
-

3. Signin API

3.1 What is Signin?

Signin verifies user credentials during login.

3.2 Why Hash Password Again?

- Stored password is hashed
 - Entered password must be hashed using the same algorithm
 - Hash values are compared
-

3.3 Signin API Code

```
router.post('/signin', (req, res) => {  
  const { email, password } = req.body  
  
  const hashedPassword = cryptojs.SHA256(password).toString()  
  
  const sql = `SELECT * FROM users WHERE email = ? AND password = ?`  
  
  pool.query(sql, [email, hashedPassword], (error, data) => {  
    if (error)  
      res.send(result.createResult(error))  
    else if (data.length == 0)  
      res.send(result.createResult('Invalid email or password'))  
    else {  
      res.send(result.createResult(null, data))  
    }  
  })  
})
```

3.4 Signin Logic

Condition	Meaning
error	Database error
data.length == 0	Invalid credentials
else	Login success

4. req.body vs req.params vs req.query

4.1. req.body

4.1.1 What is req.body?

`req.body` is used to receive **data sent inside the HTTP request body**.

It is mainly used when:

- Creating new data
- Sending sensitive data (e.g., passwords)
- Sending large or structured data (JSON)

Common HTTP Methods

- POST
- PUT
- PATCH

4.1.2 Middleware Requirement (Very Important)

```
app.use(express.json())
```

Without this middleware, `req.body` will be **undefined**.

4.1.3 Example: Sending Data Using `req.body`

Client Request (Postman – JSON Body)

```
{
  "name": "Amit",
  "email": "amit@gmail.com",
  "password": "amit123"
}
```

Express Code

```
router.post('/signup', (req, res) => {
  const { name, email, password } = req.body
  res.send(req.body)
})
```

4.1.4 When to Use `req.body`

Use Case	Reason
Signup / Signin	Sensitive data
Insert / Update	Structured data
Large payload	Body supports size

4.2. `req.params` (Request Parameters)

4.2.1 What is `req.params`?

`req.params` is used to receive **data from the URL path**.

Characteristics:

- Mandatory
- Used to identify a **specific resource**

4.2.2 URL Structure

```
/users/10
```

Here:

- `10` is a **request parameter**

4.2.3 Example: Using `req.params`

Express Route

```
router.delete('/:uid', (req, res) => {  
  const uid = req.params.uid  
  res.send(`User ID is ${uid}`)  
})
```

Postman Request

```
DELETE http://localhost:3000/user/10
```

4.2.4 When to Use `req.params`

Use Case	Reason
Delete by ID	Resource identification
Update by ID	Mandatory reference
Fetch single record	Unique identifier

4.3. `req.query` (Query Parameters)

4.3.1 What is `req.query`?

`req.query` is used to receive **optional data from the URL**.

It is mainly used for:

- Searching
 - Filtering
 - Sorting
 - Pagination
-

4.3.2 URL Structure

```
/user?email=amit@gmail.com
```

Here:

- `email` is a **query parameter**
-

4.3.3 Example: Using `req.query`

Express Route

```
router.get('/', (req, res) => {  
  const { email } = req.query  
  res.send(`Email is ${email}`)  
})
```

Postman Request

```
GET http://localhost:3000/user?email=amit@gmail.com
```

4.3.4 When to Use **req.query**

Use Case	Reason
Search	Optional filters
Filter data	Multiple conditions
Pagination	page, limit

Comparison Table (Very Important)

Feature	req.body	req.params	req.query
Data Location	Request body	URL path	URL after ?
Mandatory	✗	☑	✗
Security	High	Medium	Low
Size Limit	Large	Small	Small
Used With	POST, PUT	GET, PUT, DELETE	GET
Accessed Using	req.body	req.params	req.query

Real-Life Examples

Signup

```
POST /signup  
req.body → name, email, password
```

Delete User

```
DELETE /user/5  
req.params → uid = 5
```

Search User

```
GET /user?email=a@gmail.com  
req.query → email
```

6. Authorization Middleware

6.1 What is Middleware?

Middleware is a function that executes **before routes** and controls request flow.

6.2 Purpose of Authorization Middleware

- Allow public routes (signup, signin)
 - Block protected routes
-

6.3 Middleware Code

```
app.use((req, res, next) => {  
  const path = req.url  
  
  if (path == '/user/signin' || path == '/user/signup')  
    next()  
  else  
    res.send('You need to be authorized')  
})
```
